

Visual Companion

Building AI Application

Swarup Biswas

How FastAPI Works in This App

A beginner-friendly view of the browser, the API, and the backend flow using Chrome.

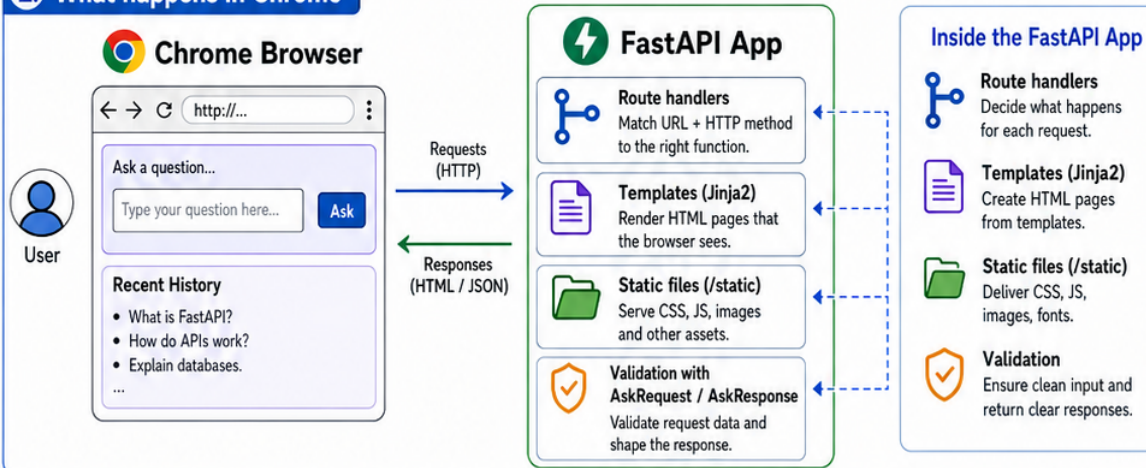
1 What FastAPI is doing



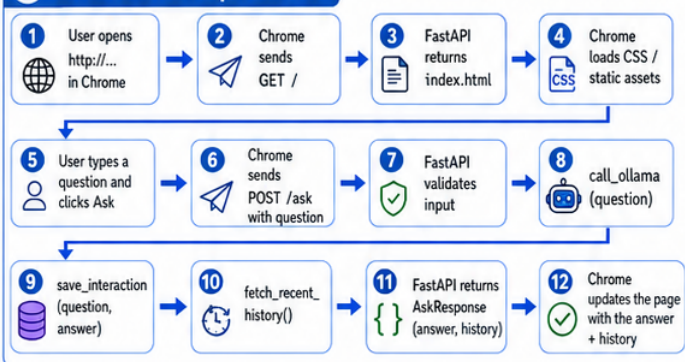
- FastAPI is the backend web application.
- It listens for browser requests.
- It decides which function should run for each URL and HTTP method.
- It returns HTML pages or JSON data back to the browser.

GET	/	-> show homepage
POST	/ask	-> process a question
GET	/history	-> return recent history
GET	/healthz	-> check system status

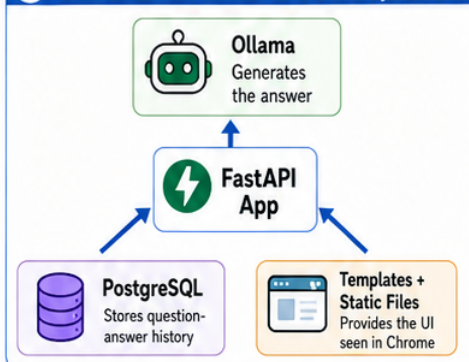
2 What happens in Chrome



3 End-to-end request flow



4 Where FastAPI connects to other systems



5 How the API is used in this app

Route	Who calls it	What it returns
GET /	Chrome	HTML page
POST /ask	Chrome form / frontend	Answer + recent history
GET /history	Browser / frontend	Saved interactions
GET /healthz	Developer / monitoring	Ollama and Postgres status

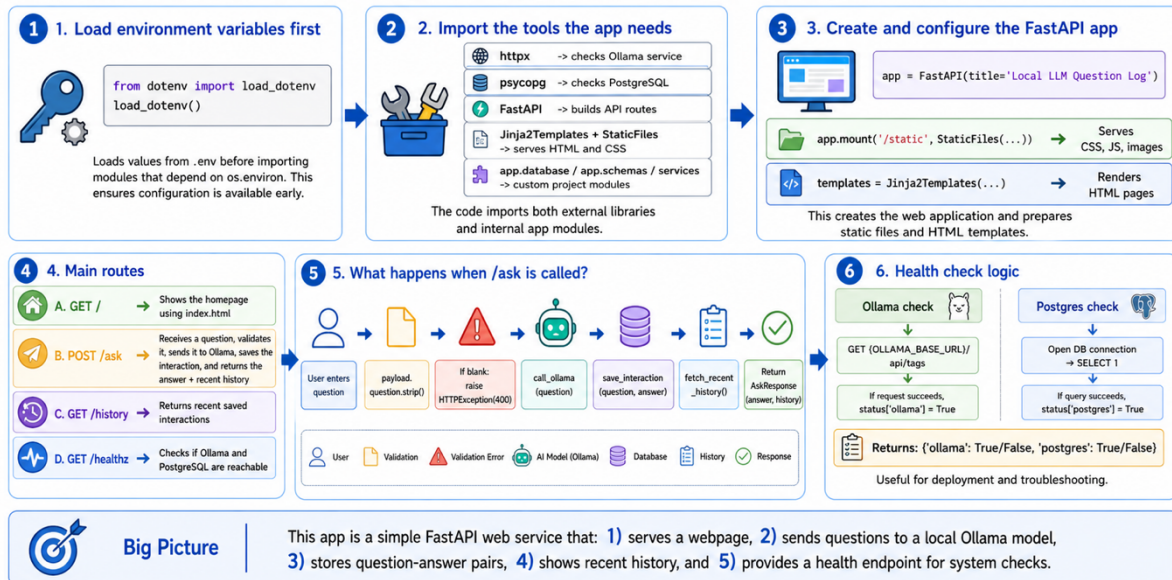


Big Picture

Chrome is the client. FastAPI is the traffic controller and backend. It serves the webpage, receives API requests, sends the question to Ollama, stores results in PostgreSQL, and sends the response back so the browser can update what the user sees.

FastAPI App Explained: Local LLM Question Log

How this Python app loads settings, serves pages, handles questions, stores history, and checks system health



How database.py, schemas.py, and main.py Work Together

A beginner-friendly visual of database connection, data models, and FastAPI request flow

1. database.py: opens a database connection

```
import os
import psycopg
from contextlib import contextmanager

DATABASE_URL = os.environ["DATABASE_URL"]

@contextmanager
def get_conn():
    with psycopg.connect(DATABASE_URL) as conn:
        yield conn
```



- Reads DATABASE_URL from environment variables
- Uses psycopg to connect to PostgreSQL
- get_conn() is a reusable helper for opening a connection
- contextmanager means the connection is cleanly opened and closed

2. schemas.py: defines the data shapes

A AskRequest

```
question: str
```

Used for incoming POST /ask data

B Interaction

```
id: int
question: str
answer: str
model_name: str
created_at: str
```

Used for saved chat history records

C AskResponse

```
answer: str
history: list[Interaction]
```

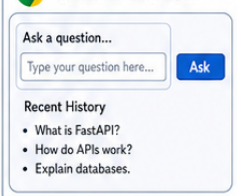
Used for the response returned by /ask



- BaseModel helps validate and structure data
- These models act like contracts for request and response data

3. main.py: the traffic controller

Chrome Browser



Requests (HTTP)
Responses (HTML / JSON)

main.py / FastAPI App

```
</> app = FastAPI(...)
app.mount('/', StaticFiles(...))
templates = Jinja2Templates(...)

GET /
POST /ask
GET /history
GET /healthz
```

database.py

get_conn()
provides a PostgreSQL connection

schemas.py

AskRequest,
AskResponse,
Interaction
shape the data



Creates the FastAPI app



Serves the homepage with Jinja2 templates



Handles API routes



Calls Ollama for answers

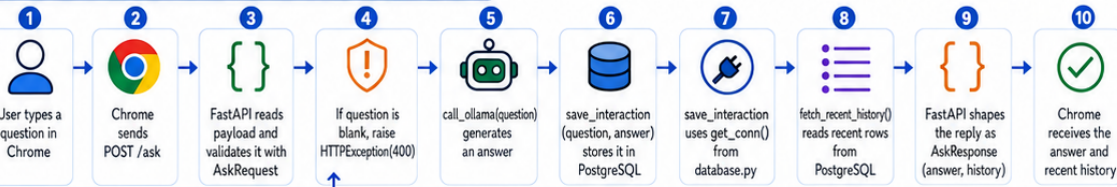


Uses database.py for database access

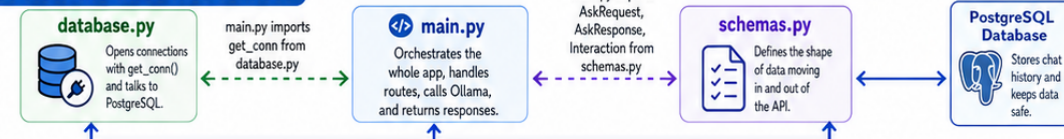


Uses schemas.py for request and response structure

4. What happens when the user asks a question?



5. How the three files connect



Big picture: database.py handles database connection, schemas.py defines the data structure, and main.py brings everything together to serve the browser, process requests, store history, and return responses.

1. THE TWO SUPPORTING FILES

A database.py – handles PostgreSQL connection

```
import os
import psycopg
from contextlib import contextmanager

DATABASE_URL = os.environ["DATABASE_URL"]

@contextmanager
def get_conn():
    with psycopg.connect(DATABASE_URL) as conn:
        yield conn
```



What it does

Reads DATABASE_URL from the environment and provides get_conn(), a safe way to get a database connection anywhere in the app.



B schemas.py – defines data shapes with Pydantic

```
from pydantic import BaseModel

class AskRequest(BaseModel):
    question: str

class Interaction(BaseModel):
    id: int
    question: str
    answer: str
    model_name: str
    created_at: str

class AskResponse(BaseModel):
    answer: str
    history: list[Interaction]
```

Pydantic =
API Inspection Gate

Pydantic checks that incoming and outgoing API data follows the structure the app promised to use.

Incoming data (Request)

AskRequest

Must contain:
question: str

Outgoing data (Response)

AskResponse

Must contain:
answer: str
history: list[Interaction]

2. HOW main.py WORKS WITH THEM (END-TO-END FLOW IN CHROME)

1 CLIENT (Chrome Browser)



User opens the app in Chrome and interacts with the web page.

What the browser sends
(when user asks a question)

```
POST /ask
Content-Type: application/json
{
  "question": "What is FastAPI?"
}
```

What the browser receives
(Example response)

```
{
  "answer": "FastAPI is a modern, fast web framework for building APIs with Python...",
  "history": [
    {
      "id": 12,
      "question": "What is FastAPI?",
      "answer": "FastAPI is a modern...",
      "model_name": "llama3.1",
      "created_at": "2024-06-01T12:34:56Z"
    },
    ...
  ]
}
```

FASTAPI APP (main.py)

- 1 User goes to /
Chrome requests GET /
FastAPI returns index.html (Jinja2 template) `@app.get("/")`
- 2 User submits a question from the page
Browser sends POST /ask with JSON body `@app.post("/ask")`
- 3 Pydantic checks incoming data with AskRequest
Must contain: question: str
If invalid → FastAPI returns 422 Unprocessable Entity `AskRequest (question: str)`
- 4 Business logic runs in ask()
• question = payload.question.strip()
• call_ollama(question) to get answer from LLM `call_ollama()`
- 5 Save the interaction to PostgreSQL
save_interaction(question, answer) `uses get_conn() from database.py`
- 6 Fetch recent history from PostgreSQL
fetch_recent_history() `uses get_conn() from database.py`
- 7 Build response data
answer + history list[Interaction]
- 8 Pydantic checks outgoing data with AskResponse
Must contain: answer: str and history: list[Interaction]
If invalid → FastAPI raises a response validation error `AskResponse (answer, history)`
- 9 FastAPI returns JSON response to the browser `Response sent`
- 10 Browser receives the JSON and updates the page
Shows answer and history to the user `User sees result`

EXTERNAL SERVICES

Ollama LLM (Local)



Endpoint:
OLLAMA_BASE_URL
/api/chat

main.py → call_ollama()
sends the question
and gets the answer.

PostgreSQL Database



Stores all
interactions
(question, answer,
model, time, etc.)

/healthz (Health Check)

main.py checks both
services:

- Calls Ollama /api/tags
- Runs SELECT 1 on PostgreSQL

Returns:
{"ollama": true/false,
"postgres": true/false}

QUICK SUMMARY

1. User in Chrome opens the page and asks a question
2. Pydantic (Inspection Gate) validates incoming data (AskRequest)
3. FastAPI runs the logic: calls LLM, saves to DB, gets history
4. Pydantic (Inspection Gate) validates outgoing data (AskResponse)
5. Browser receives response and shows answer + history

Roles at a Glance

Pydantic (schemas.py)
Defines the data contract and acts as the inspection gate for all API data.

database.py
Provides a safe, reusable connection to PostgreSQL using get_conn().

main.py (FastAPI App)
Defines routes, uses Pydantic models, connects to services, and returns responses to the browser.

★ Key Takeaway

Pydantic is not the business logic. It is the data contract checker. It makes sure the data coming in and going out matches the design.